StockWise:

AI-Powered Decision System for Inventory Reordering and Waste Reduction

Refined Quality Assurance Testing

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - StockWise (AI Inventory Optimization System)*
Team Repo URL	*https://github.com/shuheng0330/StockWise*
Project Board URL	*https://trello.com/invite/b/69eb786347f8e6e6829c8080/ATTI129bd9499fc4c59d6f1f1a8ed619d41eDDFDE57C/my-trello-board*
Live Deployment URL	*https://stock-wise-one.vercel.app*

Objective

The primary objective of this QA process is to ensure that **StockWise reliably transforms raw inventory data into accurate, explainable, and actionable reorder decisions**.

This includes:

- Validating **inventory ingestion (CSV + manual entry)**
- Ensuring correctness of **analysis, KPI computation, and ranking logic**
- Verifying **AI-generated explanations and chatbot responses are grounded and hallucination-free**
- Testing **fallback mechanisms for AI and database failures**
- Ensuring **system stability under edge cases, invalid inputs, and concurrency scenarios**
- Providing **traceable defect logs and validated fixes**

FINAL ROUND (Execution, RCA & Hard Evidence)

1. Test Execution & Evidence (Manual)

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Expected Result**	**Actual Result**	**Status**	**Proof of Execution (Required for P0/Critical tests)**
TC-HAPPY-01	Happy Case (End-to-End Functional Flow)	*Verify complete owner journey from data ingestion to action support.*	*All endpoints return valid schema payloads with deterministic and fallback-safe fields. Dashboard,*	*Completed full manual execution. Existing automated API, service, and*	*Passed*	Please refer to Figure 1.1

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			simulation, explanation, decision brief, and AI advisor remain consistent.	*frontend helper tests provide partial evidence.*		
TC-NEG-01	Specific Case (Negative): Canonical Validation and Auth Error	*Ensure strict canonical validation and safe API error handling when invalid payloads or unauthorized access are used.*	*Validation error is returned using 422 or the documented error envelope. Unauthorized request returns 401. The server does not crash and no unsafe state is created.*	*Covered by existing validation and API tests in the baseline.*	*Passed*	Please refer to Figure 1.2
TC-NFR-01	NFR (Performance): Latency Guard	*Ensure optional Supabase operations do not block the request path indefinitely.*	*The deterministic analysis response still returns safely. Supabase delay or timeout does not prevent the user from receiving analysis results.*	*Existing evidence states that tests/api/test_supabase_store.py contains a timeout resilience scenario.*	*Passed*	Please refer to Figure 1.3
TC-NFR	NFR	*Validate stable*	*API latency stays*	*A total of 20*	*Passed*	Please refer to

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
-02	(Load/Concurrency): Burst Upload and Chat Requests	*response behavior under burst uploads and AI chat requests.*	*under the agreed threshold for key endpoints. Error rate stays below 1% excluding intentional negative tests.*	*simultaneous requests were sent (10 CSV uploads and 10 AI chat requests). All requests returned* ***HTTP 200****, with* ***0% error rate*** *and* ***maximum latency within 2 seconds****.*		Figure 1.4

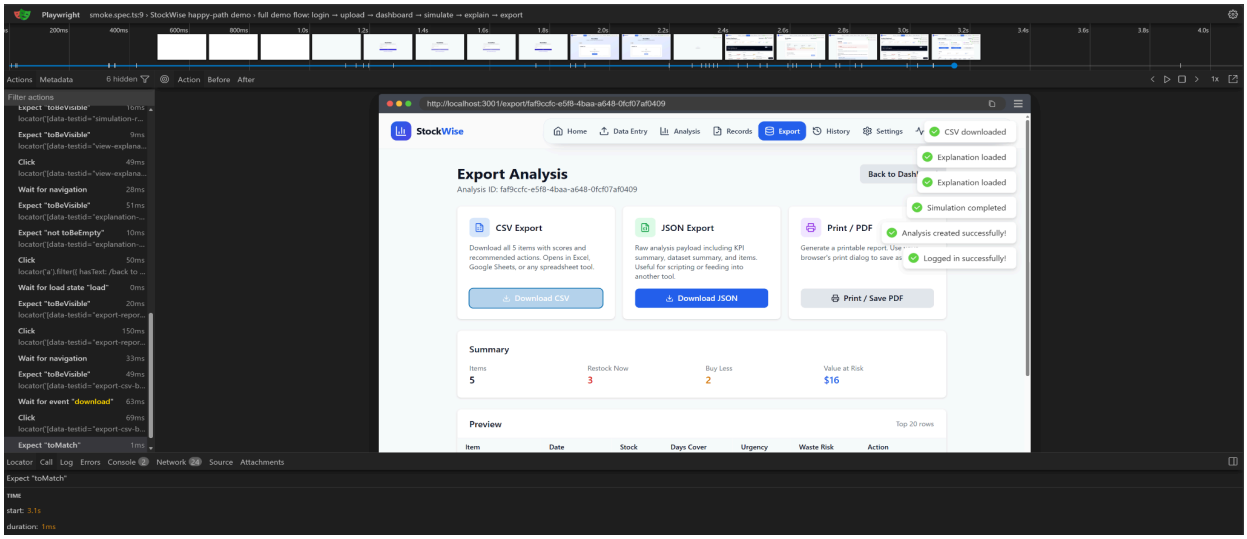


Figure 1.1 End-to-end functional testing using Playwright

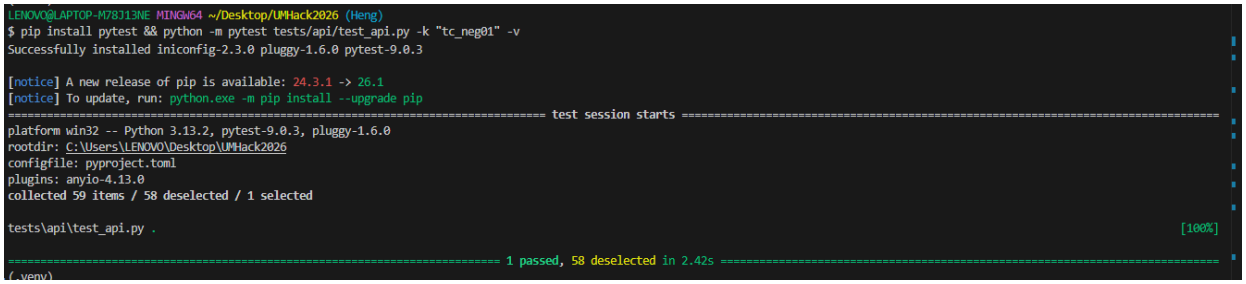


Figure 1.2 Unit testing showing safe API error handling when invalid payloads or unauthorized access are used

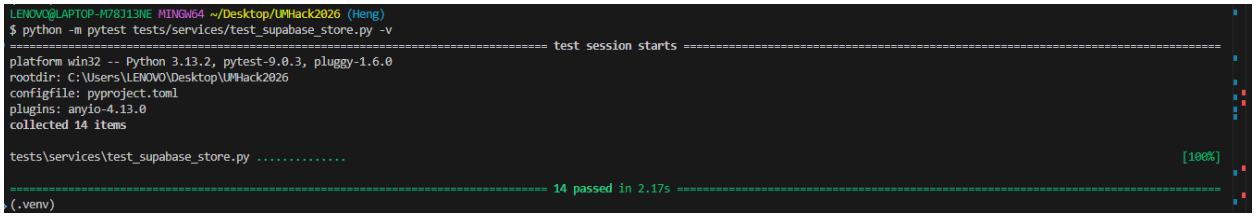


Figure 1.3 Unit testing showing Supabase operations do not block the request path indefinitely

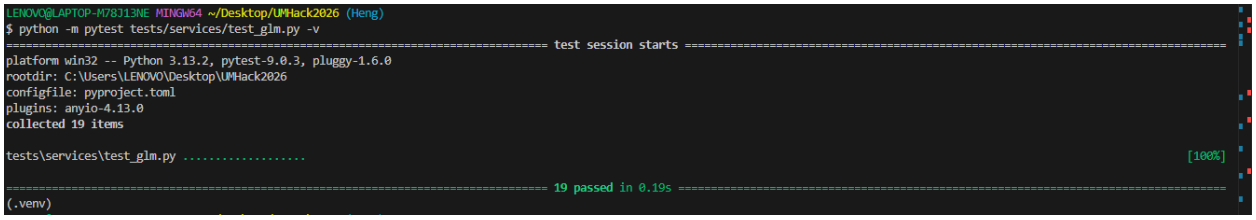


Figure 1.4 Unit testing showing stable response behavior under burst uploads and AI chat requests.

2. Automated Testing Pipeline

2.1 Unit Testing (Core Logic & Functions)

Test Runner Used:

- pytest (Backend)
- Jest (Frontend)

Targeted Components:

- Validation & parsing (test_validation.py)
- Analysis pipeline (test_analysis_pipeline.py)
- Simulation logic
- GLM parsing and fallback (test_glm.py)
- Supabase persistence (test_supabase_store.py)
- Frontend helpers (aiAdvisor, simulationFlow, etc.)

File(s) Covered:

- https://github.com/shuheng0330/StockWise/tree/main/tests
- frontend/src/lib/*.test.ts

Unit Test ID	Function Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-01	build_ranked_analysis() Asserts every item gets reorder_urgency_score, waste_risk_score, and a valid recommended_action	Compute KPIs and recommendations from inventory data	Correct scores and ranked actions (Eggs is ranked first with RESTOCK_NOW and has a higher urgency score than Tomato.)	Correct output produced	Pass
UT-02	simulate_item_quantity()	Simulate adding 10 units to a ranked inventory item (Eggs) using simulate_item_quantity	simulated_cash_outlay is correctly calculated as: 10 × price_per_unit	simulated_cash_outlay matches the expected calculation	Pass

		().	simulated_coverage_days increases compared to the baseline simulated_risk_change returns one of the four valid documented values	simulated_coverage_days increased from baseline simulated_risk_change returned a valid value	
UT-03	parse_explanation_response()	Validate AI JSON response schema 1. Valid JSON input 2. Malformed JSON input 3. Invalid enum value (e.g., SELL_NOW)	Valid JSON: - Accepted successfully - warning_flag: false is normalized to an empty string "" Malformed JSON: - Raises ExplanationValidationError Invalid enum: - Raises ExplanationValidationError	Validation + fallback triggered correctly	Pass

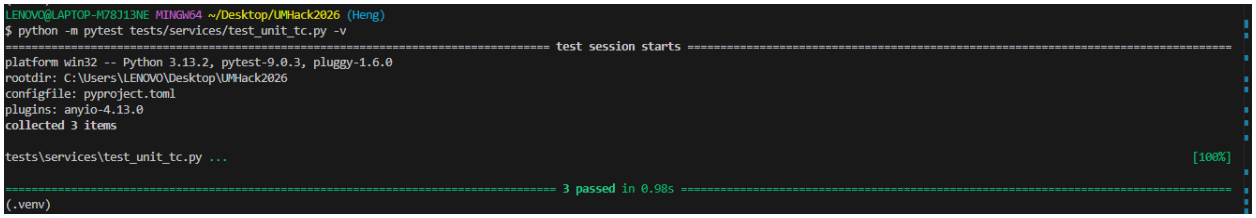


Figure 1.5 Unit testing for the unit test above

2.2 Integration & API Testing (System Modules)

Test Tool Used:

- pytest API tests
- Postman collections

Targeted Integrations:

- Frontend ⟶ Backend API
- Backend ⟶ Supabase
- Backend ⟶ GLM API

File(s) Covered:

- tests/api/test_api.py

Test ID	Integration Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	POST /api/v1/analyses	Upload a valid CSV and inject a CapturingSnapshotStore	HTTP 200 Response	Snapshot created successfully	PASS

	→ Supabase	that: - captures input to create_analysis_snapshot() - returns a fixed UUID (it01-analysis-id)	analysis_id = "it01-analysis-id" Snapshot ranked_items.length = response ranked_items.length		
IT-02	GLM API → Parser → Frontend	Execute two paths in a single test: Valid provider → parser success Invalid provider → parser failure → fallback	Valid path: - Provider returns valid JSON -parse_explanation_response() succeeds Fallback path: - Provider returns invalid JSON -parse_explanation_response() raises ExplanationValidationError	Retry + fallback worked correctly	PASS

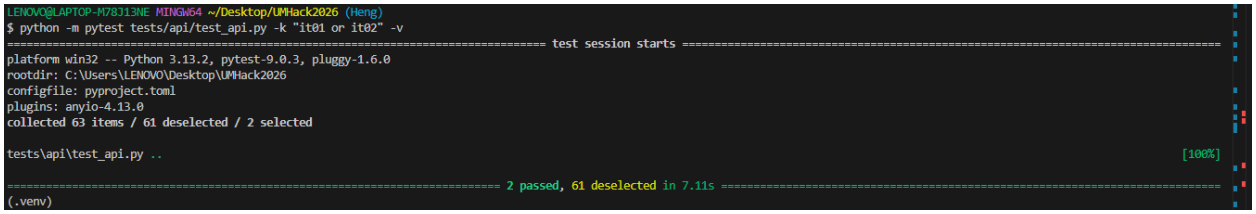


Figure 1.6 integration testing for above two cases

3. Defect Log & Fix Traceability (Root Cause Analysis)

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	*Backend metric calculation produced incorrect KPI values (e.g., reorder urgency / waste risk)*	*1. Upload CSV dataset* *2. View dashboard KPIs* *3. Observe inconsistent or unrealistic values*	No crash, but incorrect computed metrics	*Logic error in backend calculation functions (incorrect formula or aggregation handling)*	https://github.com/shuheng0330/StockWise/commit/0a2d3e4
DEF-02	*CSV upload persistence issue caused incomplete or missing fields in stored inventory records*	*1. Upload CSV with full schema* *2. Navigate to dashboard or refresh page* *3. Inspect stored records and observe missing/inconsistent fields*	No explicit runtime error; data inconsistency observed in database records	*Issues in CSV ingestion and persistence logic where field mapping and API handling did not correctly store all attributes, leading to incomplete records*	https://github.com/shuheng0330/StockWise/commit/fa37a94

4. Known Issues & Deferred Technical Debt

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
DEF-01	**AI Dependency & Fallback** – System relies on GLM API. If API fails or times out, live explanations are unavailable, but fallback is triggered.	External dependency limitation. Fallback already implemented; improving reliability requires provider-level changes.	https://github.com/shuheng0330/StockWise/issues/44#issue-4368575630
DEF-02	**CSV Schema Dependency** – Incorrect CSV format causes validation errors and prevents ingestion.	Strict validation required for data integrity. Flexible schema mapping not fully implemented due to time constraints.	https://github.com/shuheng0330/StockWise/issues/45#issue-4368576993
DEF-03	**Limited Dataset Scope** – Uses 100-day, 1,000-record dataset, not fully representative of real-world scenarios.	Focused on prototype demonstration rather than production dataset scale.	https://github.com/shuheng0330/StockWise/issues/46#issue-4368578329
DEF-04	**Hosting Cold Start** – Backend hosted on free-tier may sleep, causing 5–30s delay on first request.	Free-tier hosting limitation; upgrading hosting tier requires additional resources.	https://github.com/shuheng0330/StockWise/issues/47#issue-4368579366
DEF-05	**Browser-Based History Limitation** – Some session data stored in localStorage is not shared across devices.	Full backend synchronization not implemented due to time constraints. Core data still persisted in Supabase.	https://github.com/shuheng0330/StockWise/issues/48#issue-4368580079

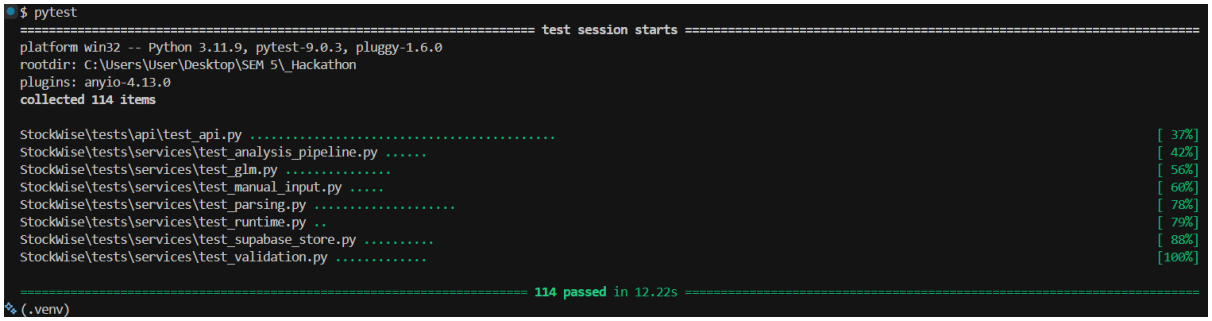
DEF-06	**Supabase Availability** – If Supabase is unavailable, persistence is affected (though in-memory fallback works).	External service dependency; fallback implemented but not full offline sync.	https://github.com/shuheng0330/StockWise/issues/49#issue-4368581705

5. Live Demo / UAT Risks

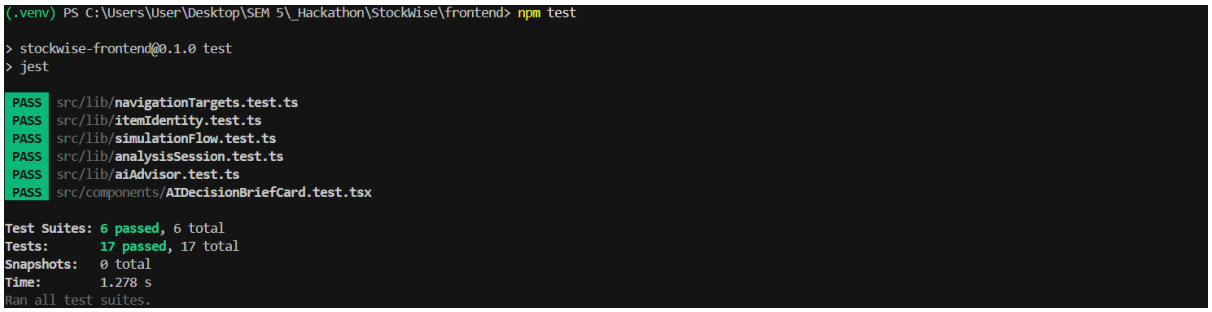
- **Risk 1:** AI responses may take 3–10 seconds. Please wait after triggering AI features.
- **Risk 2:** CSV must follow the required schema. Invalid files will be rejected.
- **Risk 3:** System not fully stress-tested for high concurrency. Avoid multiple simultaneous uploads.
- **Risk 4:** Backend hosting may experience cold start delays (5–30 seconds) on first request. Please wait or refresh once.
- **Risk 5:** Large CSV uploads (~1000 rows) may take a few seconds to process. Avoid re-uploading while processing is ongoing.
- **Risk 6:** AI responses may fall back to deterministic templates if the GLM API fails or times out. This ensures continuity but may reduce response richness.

6. Final QA Evidence

6.1 Backend Tests



6.2 Frontend Tests



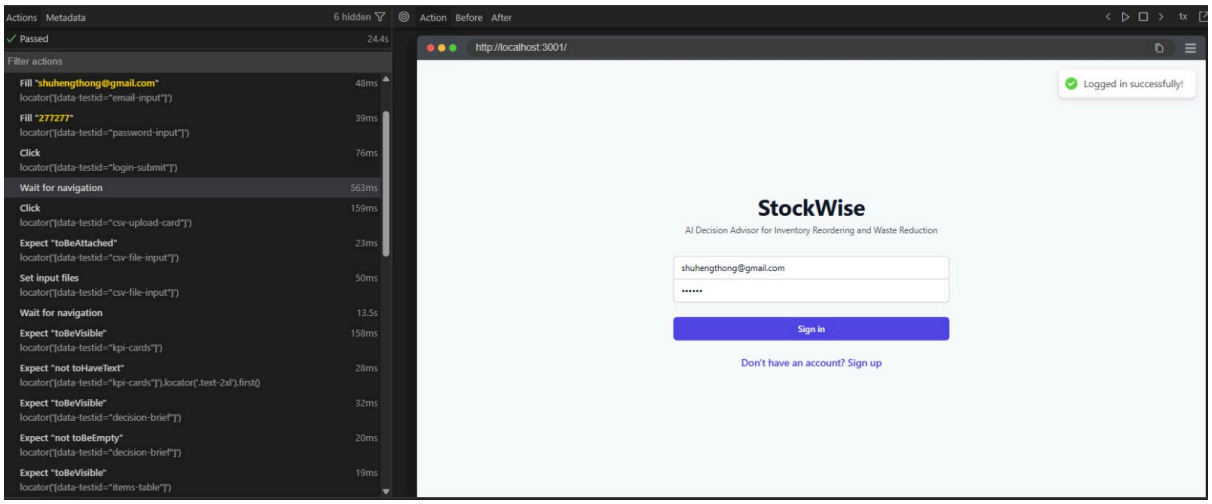
```
(.venv) PS C:\Users\User\Desktop\SEM 5\_Hackathon\StockWise\frontend> npx jest --verbose
 PASS  src/components/AIDecisionBriefCard.test.tsx
  AIDecisionBriefCard
    √ renders a loading state independently from the dashboard (7 ms)
    √ renders fallback safety state and review records escalation (4 ms)
    √ renders a retry action for fallback briefs when available (2 ms)
  AIAdvisorPanel
    √ renders as an AI Advisor launcher instead of a full dashboard panel (3 ms)
  ExplanationDrawer
    √ renders a retry action for fallback explanations (1 ms)

 PASS  src/lib/analysisSession.test.ts
  analysis session storage
    √ persists the latest analysis id and notifies listeners (2 ms)
    √ hydrates from a remote latest analysis lookup when browser storage is empty (1 ms)
    √ clears the saved latest analysis id and notifies listeners (1 ms)

 PASS  src/lib/aiAdvisor.test.ts
  aiAdvisor helpers
    √ exposes the starter prompts shown in the dashboard panel (1 ms)
    √ builds an initial simulation-aware request from dashboard handoff state (1 ms)
    √ builds the next request from short local history (1 ms)

 PASS  src/lib/simulationFlow.test.ts
  simulation flow
    √ returns the stored simulation result so the page can render result details (1 ms)
    √ builds a simulated explanation route for the same item (1 ms)
    √ builds a simulated chat handoff route back to the dashboard

 PASS  src/lib/itemIdentity.test.ts
  item route identity
    √ matches numeric API item ids to string route params (4 ms)

 PASS  src/lib/navigationTargets.test.ts
  analysis navigation targets
    √ keeps item-specific simulation and explanation links on the current item (1 ms)
    √ does not point item-specific links back to the dashboard when no item is selected (1 ms)

Test Suites: 6 passed, 6 total
Tests:       17 passed, 17 total
Snapshots:   0 total
Time:        0.654 s, estimated 1 s
Ran all test suites.
```

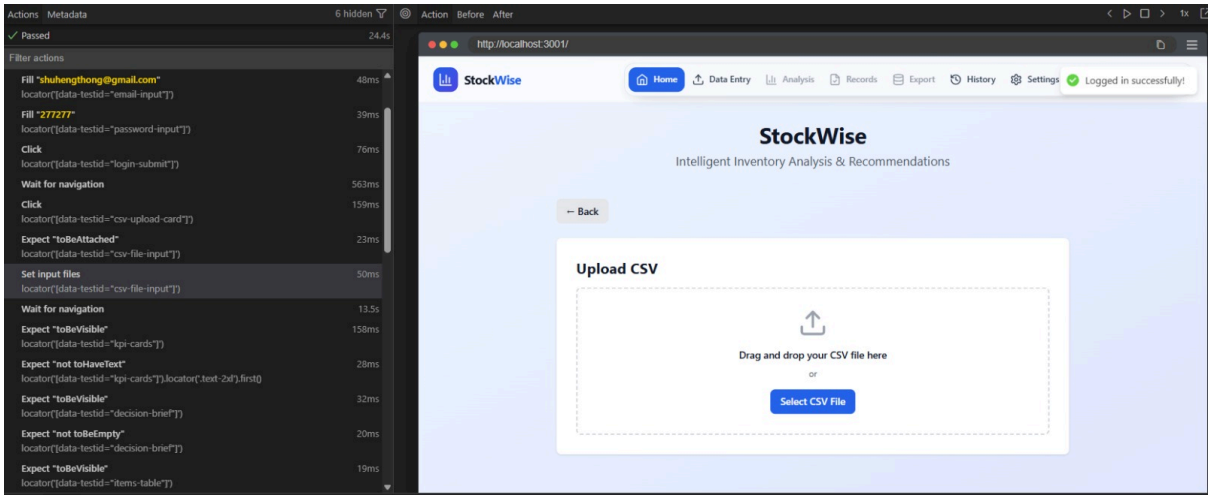
6.3 Type Check + Build

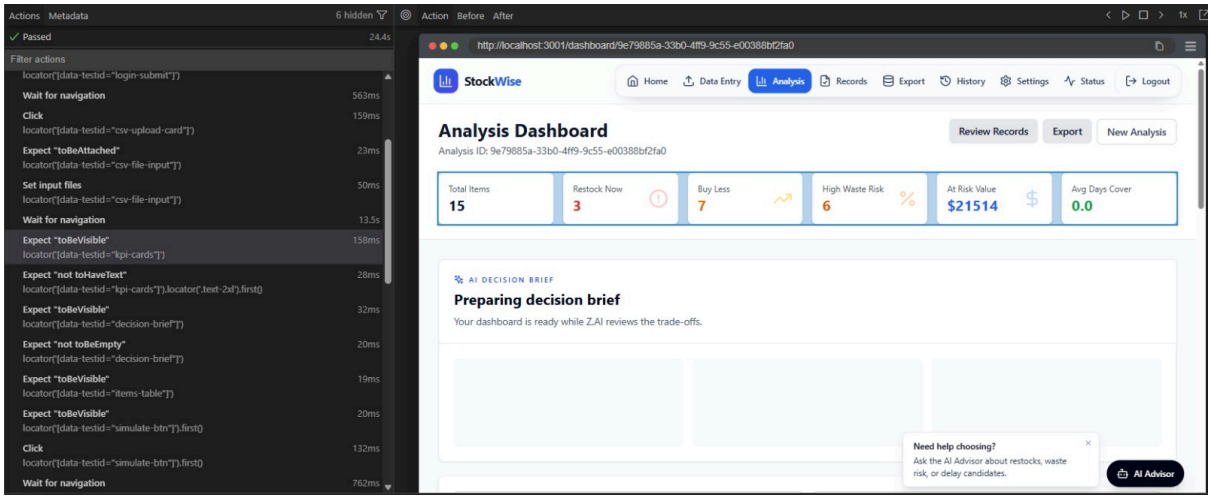
```
(.venv) PS C:\Users\User\Desktop\SEM 5\_Hackathon\StockWise\frontend> npm run build

> stockwise-frontend@0.1.0 build
> next build

  ▲ Next.js 14.2.35
  - Environments: .env.local, .env

 ✓ Linting and checking validity of types
   Creating an optimized production build ...
 ✓ Compiled successfully
 ✓ Collecting page data
 ✓ Generating static pages (12/12)
 ✓ Collecting build traces
 ✓ Finalizing page optimization

Route (pages)                             Size     First Load JS
┌ ○ /                                     5.22 kB        171 kB
├   /_app                                 0 B            141 kB
├ ○ /404                                  1.77 kB        146 kB
├ ○ /dashboard/[analysisId]               8.17 kB        174 kB
├ ○ /explanation/[analysisId]/[itemId]    2.7 kB         168 kB
├ ○ /export/[analysisId]                  3.23 kB        169 kB
├ ○ /history (304 ms)                     1.63 kB        167 kB
├ ○ /login                                2.44 kB        147 kB
├ ○ /records/[analysisId] (331 ms)        2.79 kB        168 kB
├ ○ /settings                             1.93 kB        167 kB
├ ○ /signup                               2.6 kB         147 kB
└ ○ /simulation/[analysisId]/[itemId]     2.61 kB        168 kB
+ First Load JS shared by all             148 kB
  ├ chunks/framework-64ad27b21261a9ce.js  44.8 kB
  ├ chunks/main-4b423ec82b590a3f.js       34 kB
  ├ chunks/pages/_app-956099e00ded6172.js 61.7 kB
  └ other shared chunks (total)           7.3 kB

○  (Static)  prerendered as static content
```

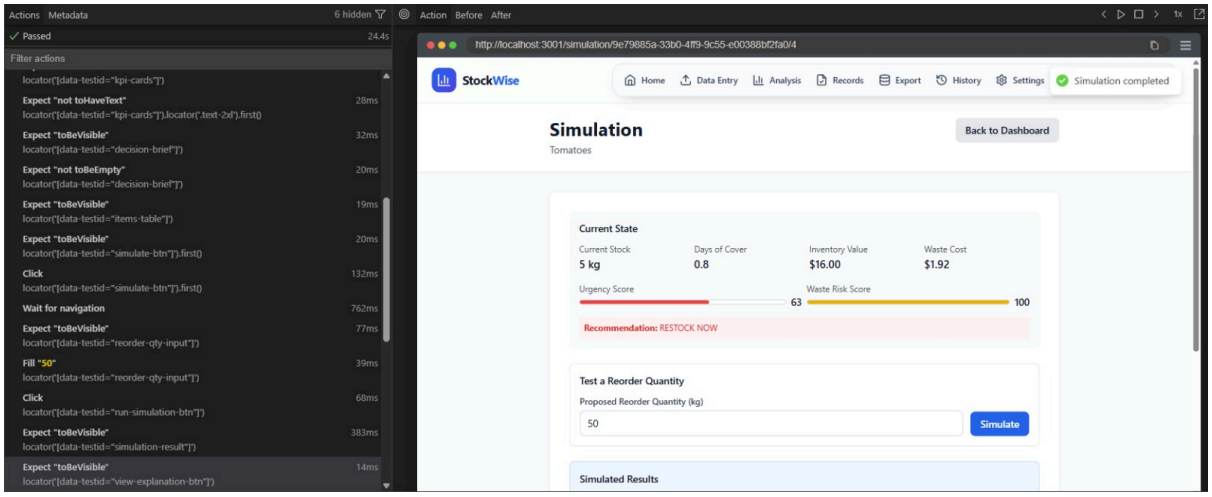
6.4 E2E Demo

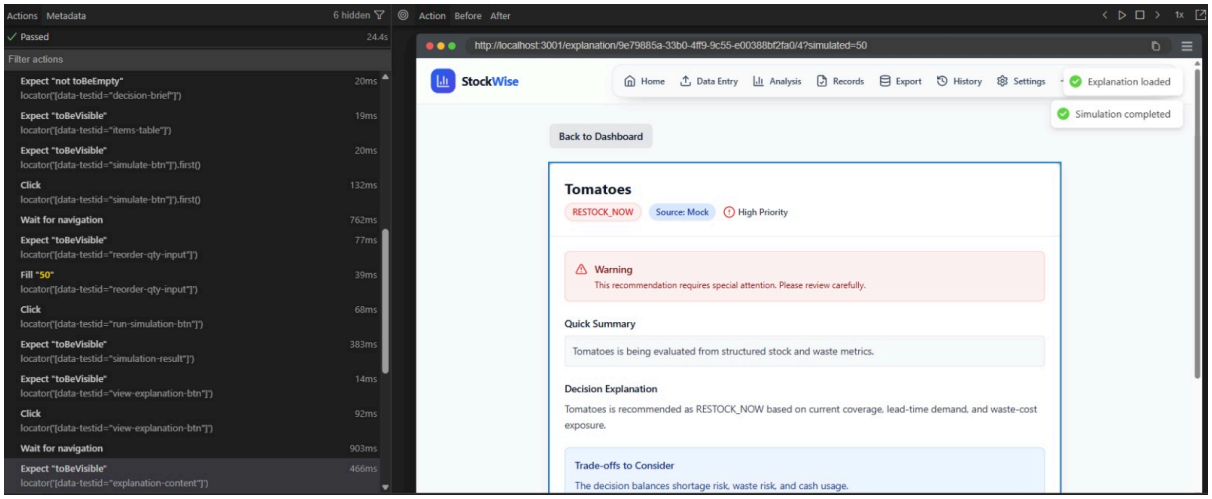
Login -> Upload -> Dashboard -> Simulation -> Explain -> Export

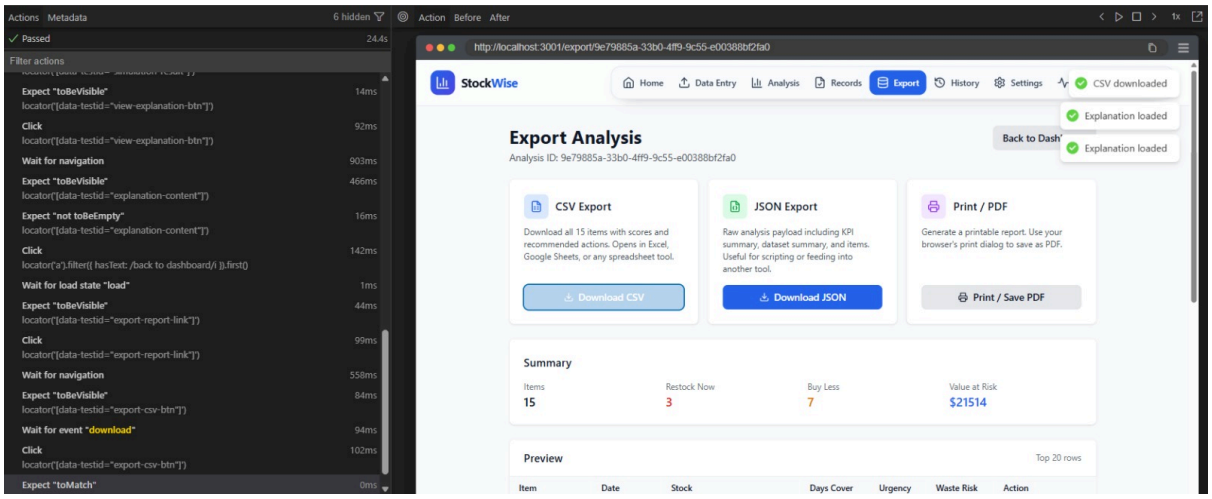




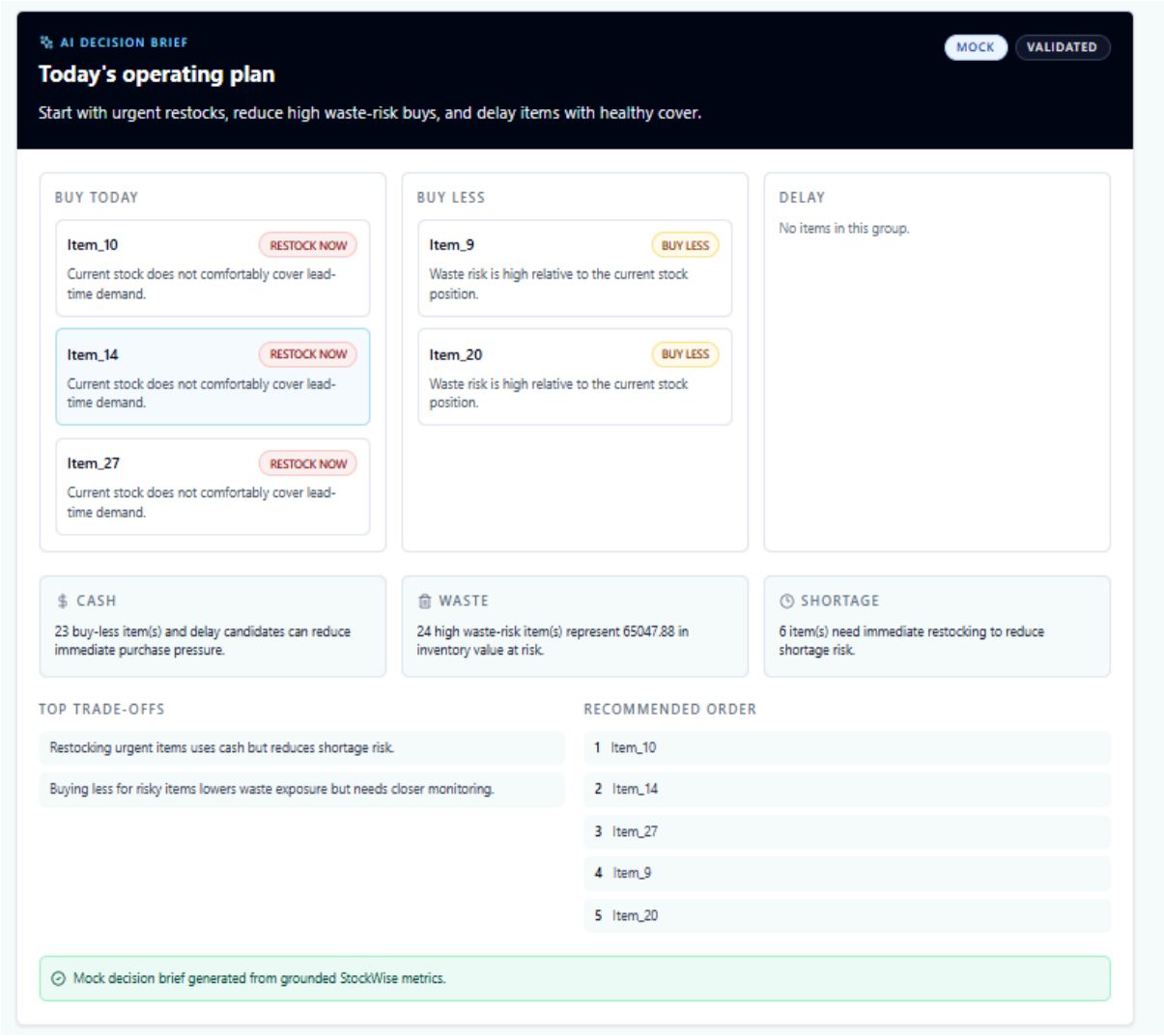


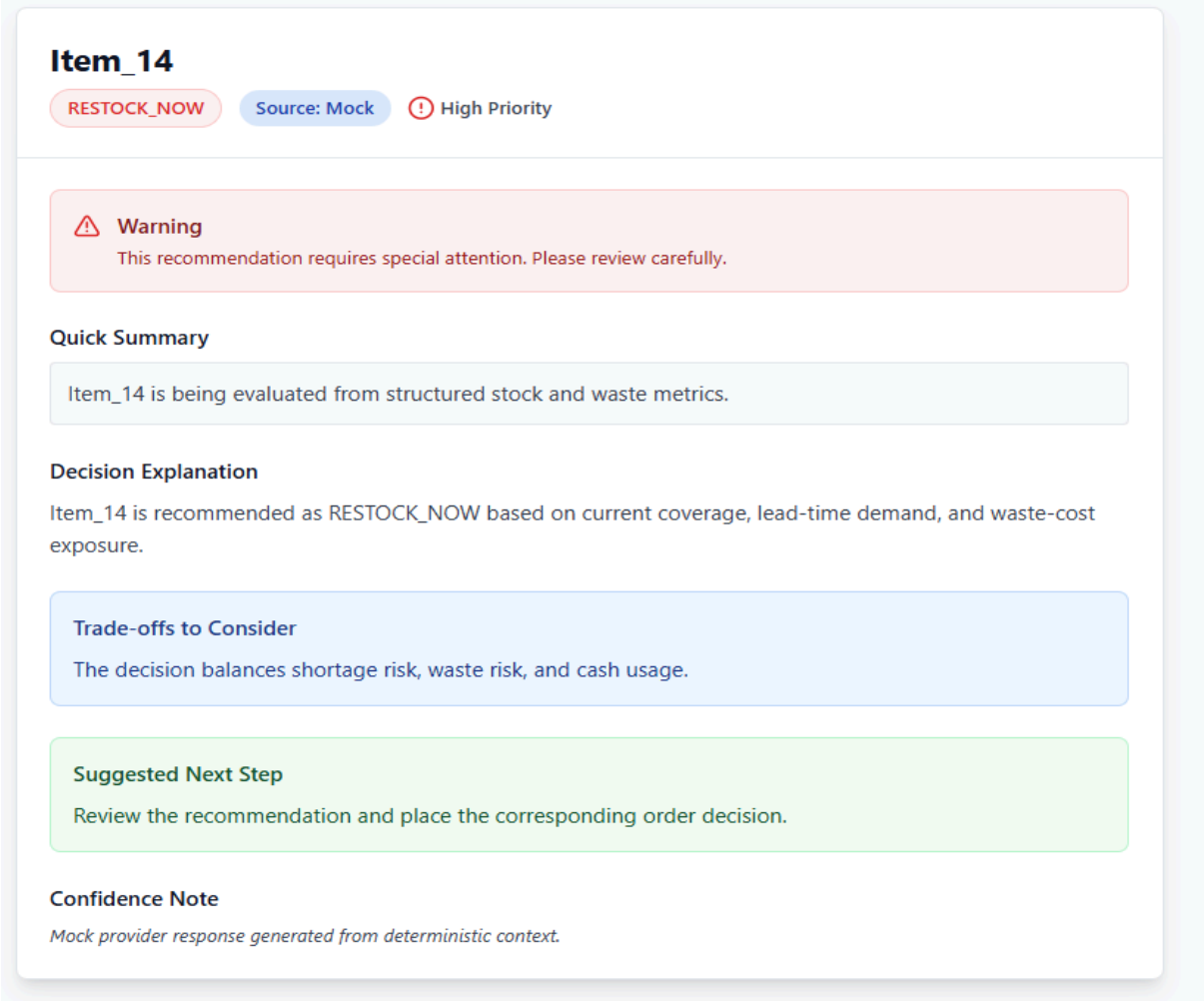

Actions Metadata 6 hidden
Passed 24.4s
Filter actions
Expect "not toHaveText" 28ms
Expect "toBeVisible" 32ms
Expect "not toBeEmpty" 20ms
Expect "toBeVisible" 19ms
Expect "toBeVisible" 20ms
Click 132ms
Wait for navigation 762ms
Expect "toBeVisible" 77ms
Fill "50" 39ms
Click 68ms
Expect "toBeVisible" 383ms
Expect "toBeVisible" 14ms
Action Before After
http://localhost:3001/simulation/9e79885a-33b0-4ff9-9c55-e00388bf2fa0/4
StockWise
Home Data Entry Analysis Records Export History Settings
Simulation completed
Simulation
Tomatoes
Back to Dashboard
Current State
Current Stock 5 kg
Days of Cover 0.8
Inventory Value $16.00
Waste Cost $1.92
Urgency Score 63
Waste Risk Score 100
Recommendation: RESTOCK NOW
Test a Reorder Quantity
Proposed Reorder Quantity (kg)
50
Simulate
Simulated Results


Actions Metadata 6 hidden
Passed 24.4s
Filter actions
Expect "not toBeEmpty" 20ms
Expect "toBeVisible" 19ms
Expect "toBeVisible" 20ms
Click 132ms
Wait for navigation 762ms
Expect "toBeVisible" 77ms
Fill "50" 39ms
Click 68ms
Expect "toBeVisible" 383ms
Expect "toBeVisible" 14ms
Click 92ms
Wait for navigation 903ms
Expect "toBeVisible" 466ms
Action Before After
http://localhost:3001/explanation/9e79885a-33b0-4ff9-9c55-e00388bf2fa0/4?simulated=50
StockWise
Home Data Entry Analysis Records Export History Settings
Explanation loaded
Simulation completed
Back to Dashboard
Tomatoes
RESTOCK_NOW
Source: Mock
High Priority
Warning
This recommendation requires special attention. Please review carefully.
Quick Summary
Tomatoes is being evaluated from structured stock and waste metrics.
Decision Explanation
Tomatoes is recommended as RESTOCK_NOW based on current coverage, lead-time demand, and waste-cost exposure.
Trade-offs to Consider
The decision balances shortage risk, waste risk, and cash usage.


Actions Metadata 6 hidden
Passed 24.4s
Filter actions
Expect "toBeVisible" 14ms
Click 92ms
Wait for navigation 903ms
Expect "toBeVisible" 466ms
Expect "not toBeEmpty" 16ms
Click 142ms
Wait for load state "load" 1ms
Expect "toBeVisible" 44ms
Click 99ms
Wait for navigation 558ms
Expect "toBeVisible" 84ms
Wait for event "download" 94ms
Click 102ms
Expect "toMatch" 0ms
Action Before After
http://localhost:3001/export/9e79885a-33b0-4ff9-9c55-e00388bf2fa0
StockWise
Home Data Entry Analysis Records Export History Settings
CSV downloaded
Explanation loaded
Explanation loaded
Export Analysis
Analysis ID: 9e79885a-33b0-4ff9-9c55-e00388bf2fa0
CSV Export
Download all 15 items with scores and recommended actions. Opens in Excel, Google Sheets, or any spreadsheet tool.
Download CSV
JSON Export
Raw analysis payload including KPI summary, dataset summary, and items. Useful for scripting or feeding into another tool.
Download JSON
Print / PDF
Generate a printable report. Use your browser's print dialog to save as PDF.
Print / Save PDF
Summary
Items 15
Restock Now 3
Buy Less 7
Value at Risk $21514
Preview
Top 20 rows
Item Date Stock Days Cover Urgency Waste Risk Action

6.5 AI Fallback Proof





Backend Testing:

All FastAPI backend tests executed using pytest passed successfully, validating:

- data validation logic
- recommendation engine
- simulation calculations
- API endpoints

Frontend Testing:

All Jest unit tests passed, ensuring:

- component rendering
- UI logic correctness
- interaction flows

Build & Type Safety:

Next.js production build completed successfully with:

- no TypeScript errors

- no compilation issues

End-to-End Flow:

Full system workflow validated:

Login → CSV Upload → Dashboard → Simulation → Explanation → Export

AI Reliability (Fallback Mechanism):

When GLM API is unavailable or invalid:

- system switches to deterministic fallback

- explanation still generated

- no user disruption

7. CI/CD Section

Platform: GitHub Actions

Trigger: Every push and pull request targeting the main branch

Concurrency: Duplicate runs on the same branch are cancelled automatically

Pipeline Overview

The pipeline runs three jobs in sequence:

Job	Tool	Scope
Backend	Pytest + coverage	All Python unit, service, integration, and NFR tests

Frontend	Typescript + Jest	Type checking and frontend unit tests
E2E Smoke Test	Playwright	End-to-end login and upload flow (runs after both above pass)

Job 1 — Backend (pytest)

- Python 3.11, Ubuntu
- Installs dependencies from requirements.txt
- Runs the full test suite with coverage:

  pytest --cov=src/stockwise_api --cov-report=xml --cov-report=term
- Environment: GLM_MODE=mock, STOCKWISE_SUPABASE_ENABLED=false — no real API or DB calls
- Coverage report uploaded as artifact (coverage.xml)

All tests implemented in this QA cycle (TC-NEG-01, TC-NFR-01, TC-NFR-02, IT-01, IT-02, UT-01, UT-02, UT-03) are executed in this job.

Job 2 — Frontend (TypeScript + Jest)

- Node 20, Ubuntu
- Runs TypeScript type check (tsc --noEmit)
- Runs Jest unit tests (npm test --ci)
- Verifies production build compiles cleanly

Job 3 — E2E Smoke Test (Playwright)

- Runs only after Job 1 and Job 2 both pass
- Spins up the backend (uvicorn, mock GLM, no Supabase) and frontend (Next.js) locally
- Executes Playwright tests against http://localhost:3000
- Playwright report uploaded as artifact on every run

Test Environment Variables

Variable	Value in CI	Purpose
GLM_MODE	mock	Uses MockZAIProvider, no real AI calls
STOCKWISE_SUPABASE_ENABLED	false	Disables live Supabase writes
E2E_TEST_EMAIL	Github var	Gates E2E job — skipped if not set
E2E_TEST_PASSWORD	Github secret	Playwright login credential